Workshop 6

Google Play

Table of Contents

Motivation

- Your app will be ready soon!
  - (We surely hope so.)
- We want to share it with other people.
- In order to do that, we need to convert our project files into an **executable** file, which can be **distributed easily** without the need to understand how app executables work.
- => **Uploading the app to Google Play**.

What do we have to do?

- **Set up the new version**
  - Updating version number and review our Manifest file.
- **Make our app more appealing**
  - Prepare graphics: icon, artwork…
- **Handle legal issues**
  - Copyrights, Privacy Policy & TOS.
- **Keep it secure**
  - Create a signing key for app releases.
- **Create an executable**
  - Compile our app into AAB & APK files and test them.
- **Publish it!**
  - Upload your app to Google Play Console.

Package Name

- Every app has a **unique package name**, an identifier for the app on the Google Play store.
- It is also used as a unique identifier for your app in third party applications.
  - Used to connect your app with services such as Firebase and Facebook.
- Some package name prefixes are **reserved domains.**
  - Examples: com.android, com.google, android, com.example
- In our course, all apps must use the **com.technion** prefix.
- Complicated to change!
  - If you didn't use the requested format, Better call Flutter ☺️
    - Check out **change_app_package_name** in pub.dev.

App Manifest

- The **App Manifest** file describes essential information about your app to the Android build tools, the Android OS and Google Play.
- It is located in `[project]/android/app/src/main/`**`AndroidManifest.xml`**.
- Before compiling your project for release, you should check that the values in this file are correct, especially the following:
  - In the `<application>` tag, the **`android:label`** attribute defines the final name of the app.
  - Make sure that you reference all required **`<uses-permission>`** tags, otherwise your app won't work.
    - **Important!** If your app needs internet access (which is probably the case), you must reference the **`android.permission.INTERNET`** permission. It works without it during the development process but not when the app is released!

App Versions

- We've already discussed the importance of **versioning** practices.
  - Reminder: DB versioning in Workshop 5.
- It gives the **user** more information about the current version and about the upgrade versions available.
- It also allows **other apps** to determine compatibility.
- It also gives **you (the developers)** some context in the long-run.

App Versions (cont.)

- Each version is defined by 2 variables:
  - **versionCode**: 1, 2, 3…
    - A positive integer used as an **internal** version number.
    - **Motivation**: The Android system uses it to protect against downgrades.
      - How?
    - Higher numbers = more recent versions.
    - Does not indicate the significance of the updated release.
    - You can't upload an APK/AAB to the Play Store with a previously used versionCode.
    - **Convention:** Start with 1 and increment by one in each release.

App Versions (cont.)

- Each version is defined by 2 variables:
  - **versionName**: 1.0.0, 1.3.7, 2.0.5…
    - A string used as the version number shown to **users.**
    - **Motivation**: indicate to the users the significance of the updated release.
    - The format should follow **<major>.<minor>.<point>**.
    - **Convention** (semver): You should increment the
      - **Major** value when you make groundbreaking, incompatible API changes
      - **Minor** value when you add new backward-compatible features.
      - **Point** value when you make bug fixes, small code changes that require users to upgrade.
    - For sprint 1, you are required to use **1.x.x**, and for sprint 2 you are required to use **2.x.x**.

App Versions (cont.)

- In Flutter apps, we can easily change the app's version.
- In your **`pubspec.yaml`** file, update the `version` line with:

```
version: <versionName>+<versionCode>
```

- **Important**: Remember to update your version before compiling your app, as it is required when uploading the executable to Google Play!

Copyrights

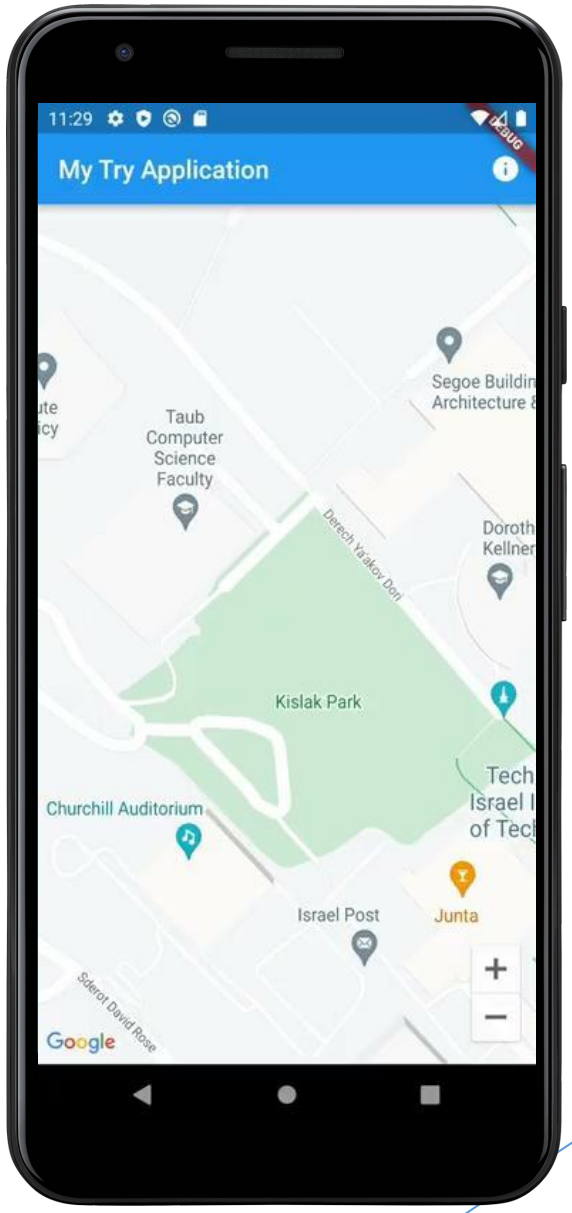
- Every piece of media, text of third-party source that you use in your app and was not originally created by you – requires you to have a **Creative Commons (CC)** copyright license.
- Each Flutter package used in your code also requires a license!
- Google requires you to give the creators credit – in your application – according to the appropriate licenses.
- You should keep track of all your used sources on-the-go!

Copyrights - AboutDialog

- Flutter provides us with **built-in Material components** which automatically generate a list of all license files used in your app.
- The built-in function **`showAboutDialog`** simply shows an “about” dialog box, which can contain the app’s icon, name, version and copyright, in addition to a “Show Licenses” button.
- This button automatically calls another built-in function called **`showLicensePage`**, which displays a page with the software licenses used by the application (all packages used…).
  - You might need to add a few licenses by yourself for media and text.
- Design a suitable location for the about dialog, even if it’s not implemented yet!

AboutDialog (Example)

```
Scaffold(
  appBar: AppBar(
    title: const Text(“My Try Application”),
    actions: [
      IconButton(
        icon: const Icon(Icons.info),
        onPressed: () {
          showAboutDialog(context: context);
        }
      ),
    ],
  ),
  body: …
)
```



Launcher Icon

- An app icon allows you to differentiate your app by using a distinct style and appearance. It should match your app's branding, colors and look & feel.
- The **Launcher icon** represents the application on the device's home screen.
  - What is the default launcher icon in Flutter apps?
- Design conventions vary with platform, OS version, etc.
  - You are expected to try and follow the Material Design guidelines as much as possible.




Launcher Icon (cont.)

- The **flutter_launcher_icons** package simplifies the task of updating the launcher icon and provides flexibility.
  - In your project's root, create a new config file called **flutter_launcher_icons.yaml**.

```
flutter_icons:
  android: true
  ios: true
  image_path: "path/to/icon.png"
```

Override the default existing Flutter launcher icon

  - In your console, install the package and then run the following command:

```
flutter pub run flutter_launcher_icons
```

  - The icon will be automatically updated in all related files!
  - Check out the (Unofficial) Android Asset Studio – it's great for generating icons!

App Signing - Background

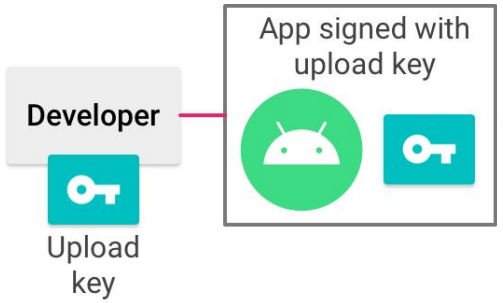
- Can you find a **security problem** with the uploading process?
- What if an attacker has your project code?
  - E.g., your Git repository is public.
  - He’s able to change the code and then compile the app by himself.
  - **Malicious updates!**
- We must be able to make sure that the suggested update for the application was made by the app developers!
  - **Verify the identity!**
- In addition, 3rd party services must be able to make sure that they contact the verified application, so that privacy is kept.

App Signing – Background (cont.)

- Verification can be done by **encrypting** the app files, using **secure and safe keys** that are only the actual developers (and Google Play) have.
- In practice, each verification is done using pairs of public\private keys. Each pair is contained securely in a single file called a **certificate**.
- The certificate also contains some metadata identifying its owner, such as his/her name, organization, etc.
- We use this pair of keys, with some advanced encryption algorithm (such as RSA), to encrypt the app files which are later uploaded to Google Play.
- The process of encrypting the app files with the keys is called **app signing**.

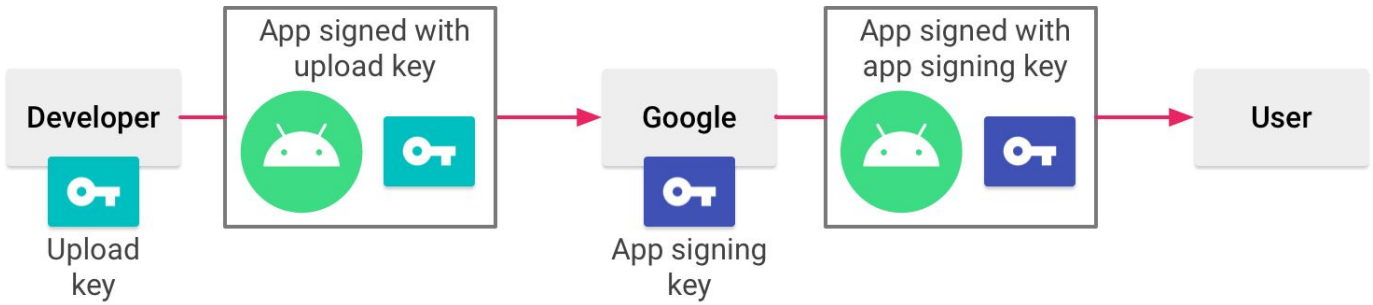
Keystores

- For signing an app, we use **Java Keystore** files (**.jks**).
- Two types of keys are used in the process:
  - **Upload key** is generated by **you** when you compile your project. It is used to verify your identity when uploading updates.
    - Generated **once**, and then used for all app updates. Make sure to keep it safe and private and not lose it, otherwise you won't be able to update your application.
      - **Do not check it into Git!**
    - You must provide a **keystore password** and an **alias (key) password** – pick strong passwords and make sure you remember them!



Keystores (cont.)

- For signing an app, we use **Java Keystore** files (**.jks**).
- Two types of keys are used in the process:
  - **App Signing Key** is generated by **Google Play Console** when you upload your app. It is used to verify the app for 3rd party services, as well as for signing the platform-specific executables (Will be explained later).
    - Generated **once**, kept on Google's secure infrastructure.
    - You might have to use the **certificate fingerprint** – a unique representation of the App Signing certificate (MD5, SHA-1 and SHA-256) in some of the services used by the application, such as Firebase (Will be explained later).



Generating an upload key

- The key generation is done with the **keytool** file, which is saved in your Java binary path. **The generation process isn't related to your project!**
  - **Tip:** run `flutter doctor -v` and find the path in the "Java binary at" line.
- Run the following line in your console:
  - **Windows:**

```
keytool -genkey -v -keystore <PATH>\<NAME>.jks -storetype JKS
-keyalg RSA -keysize 2048 -validity 10000 -alias upload
```

  - **Mac/Linux:**

```
keytool -genkey -v -keystore <PATH>/<NAME>.jks -keyalg RSA
-keysize 2048 -validity 10000 -alias upload
```

- It generates a long-term .jks file with an "upload" alias.
  - Keep it **private** and don't check it into Git!

Referencing the keystore from the app

- We've generated an update keystore, and now we want to use it inside our project for the compilation process.
- In your `[project]/android` folder, create a file called **`key.properties`** which contains a reference to your keystore:

```
storePassword=<keystore password from previous step>
keyPassword=<alias (key) password from previous step>
keyAlias=upload
storeFile=<path to your .jks file>
```

- The `key.properties` file contains secret passwords which protect your keystore. Therefore, **keep it private** and don't check it into git!
  - Your `.gitignore` file might already reference this secret file.

Configuring signing in gradle (1/2)

- The `[project]/android/app/`**`build.gradle`** file includes instructions for an automated build of your app. We want to configure it such that when compiling the project, we create encrypted app files.
- First, we extract our keystore information from the properties file and load it into a Properties object, just before the `android` block:

```
def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
  keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}

android {
  ...
}
```

Configuring signing in gradle (2/2)

- The `buildTypes` block defines the configuration for each build type:

```
buildTypes {
  release {
     signingConfig signingConfigs.debug
  }
}
```

- Replace it with the updated signing conf. info, and run `flutter clean`:

```
signingConfigs {
  release {
     keyAlias keystoreProperties['keyAlias']
     keyPassword keystoreProperties['keyPassword']
     storeFile keystoreProperties['storeFile']
          ? file(keystoreProperties['storeFile']) : null
     storePassword keystoreProperties['storePassword']
  }
}
buildTypes {
  release {
     signingConfig signingConfigs.release
  }
}
```

APK & AAB

- Our code is ready, the basic configurations (Manifest, launcher, etc.) are set, and the signing and building processes are configured. It's time to create our app files – **the executables!**
  - (well, kind of).
- We use 2 different file types:
  - **APK (Android Package Kit)** is a **packaging file**, which contains all the elements that an app needs to install correctly on your specific device. This is the file that is installed on your device when you install an app.
  - **AAB (Android App Bundle)** is a **publishing file**, which is used by Google to generate and serve optimized APKs for each user's device configuration, so that unnecessary code and resources are not downloaded => Smaller APK files.

APK & AAB (cont.)

- You should compile your project into an **AAB file**, which is then uploaded to Google Play. This can be done with:

```
flutter build appbundle
```

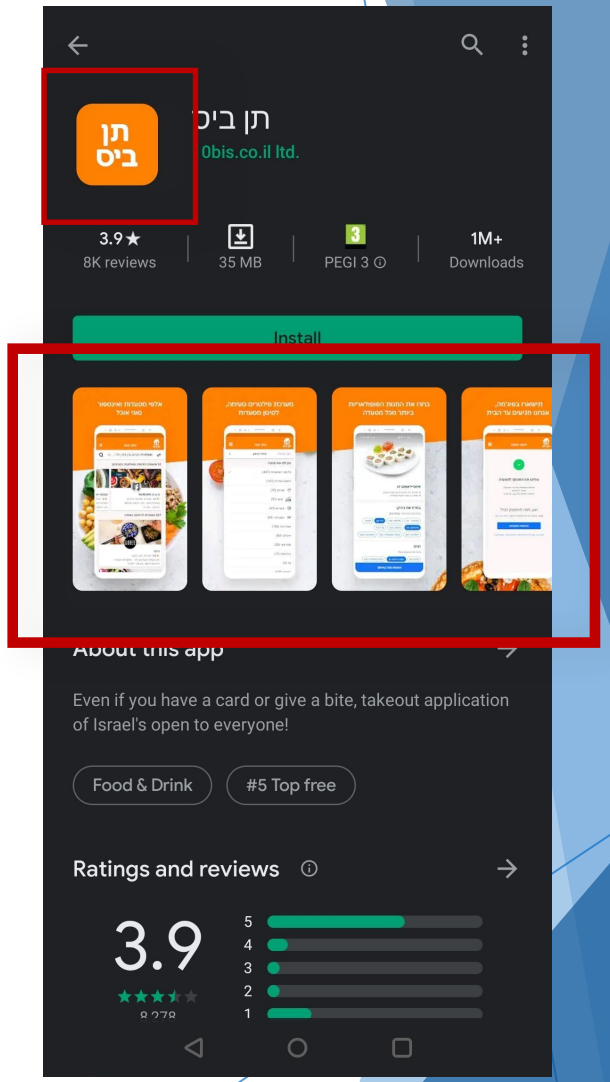
- When users install your app, Google Play will generate an appropriate **APK file** (lazily) according to the content of the original AAB file.
  - You can compile your project into APK format by yourselves with:

```
flutter build apk
```

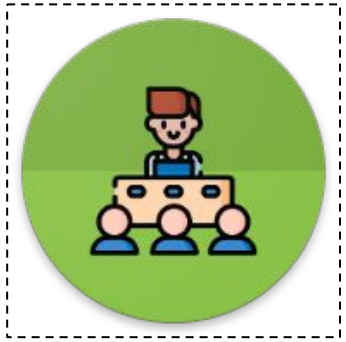
  - When you submit your app in the course website at the end of the sprint, you should submit an APK file.
- APKs / AABs that were uploaded to Google Play cannot be deleted, and you cannot start over the upload process in any case!

Graphics

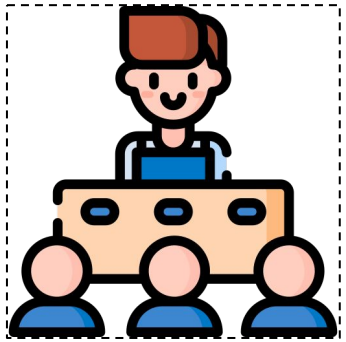
- Every app submitted to the Play Store must include a minimal set of graphics:
  - 512x512 **icon** (**without** rounded corners and transparent backgrounds!)
    - This is not the same as the launcher icon!
  - At least 2 **screenshots** of the app.
  - 1024x500 **feature graphic** (for older Play Store UI).
  - Everything else is optional and is up to you!



Graphics - Example

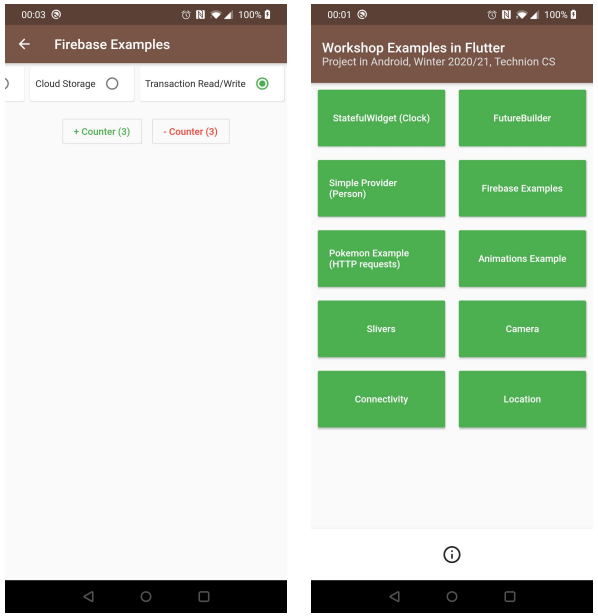


Launcher icon



512x512 icon

Feature graphic



Screenshots

Privacy Policy & Terms of Service

- Following GDPR laws of 2018, Google requires **all** submitted apps to include the following:
  - **A Privacy Policy** – Containing information about how the app interacts with and collects data from your users.
  - **A Terms of Service** agreement.

- **Important Note**: Google will not approve your app if you don't fulfill this requirement.

Privacy Policy & Terms of Service (cont.)

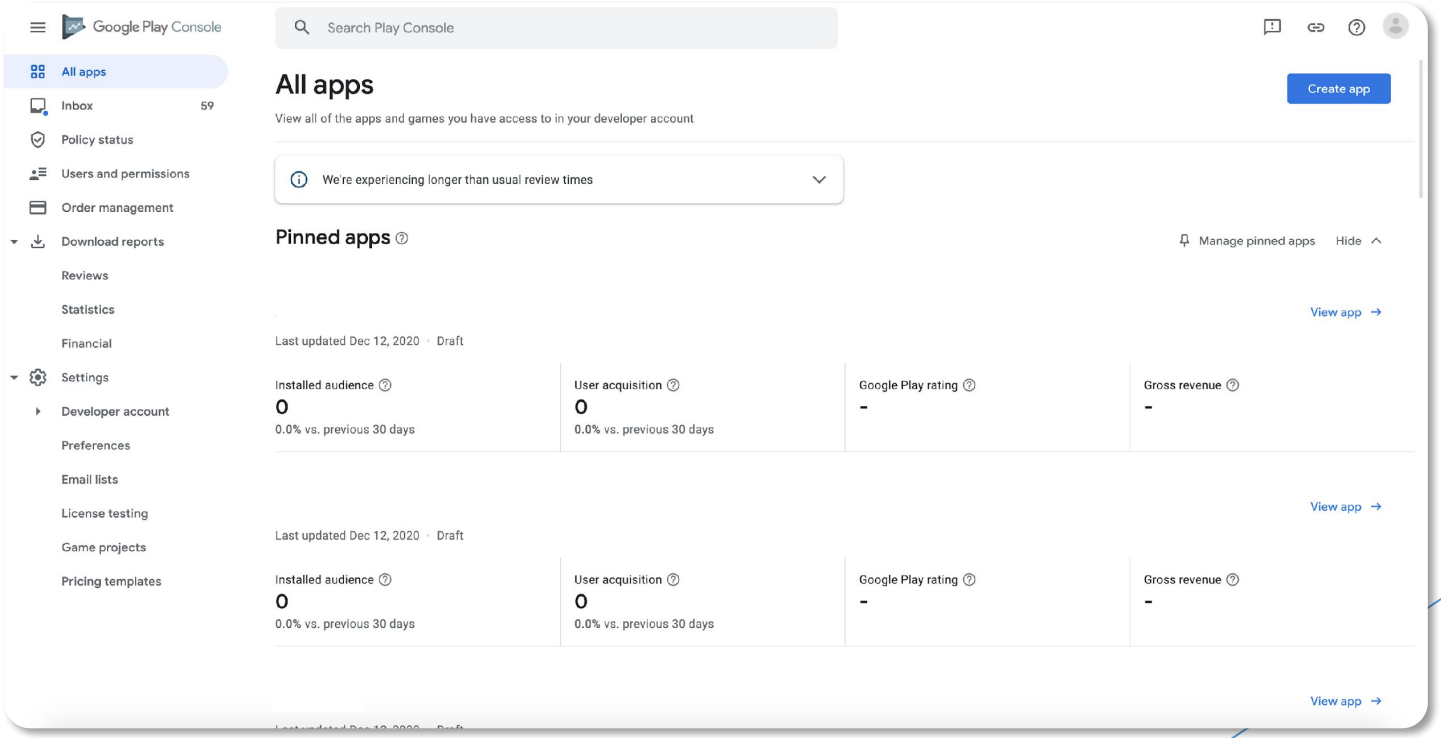
- Luckily, we don't have to write these legal documents by ourselves.
- Use **this website** to generate both in **Markdown (.md)** format:

  https://app-privacy-policy-generator.firebaseapp.com/

  - **Contact Information** – Include an email of one of the team members.
  - **App Type** – If your app contains ads, select "Ad supported". Otherwise, select "Free".
  - **Owner Type** – Individual.
- When you're done, copy the generated Markdown into two Gists (static Git files) and name them **<AppName>-PrivacyPolicy.md** and **<AppName>-TOS.md** respectively. Later, you'll use the Gists links in your Google Play upload.

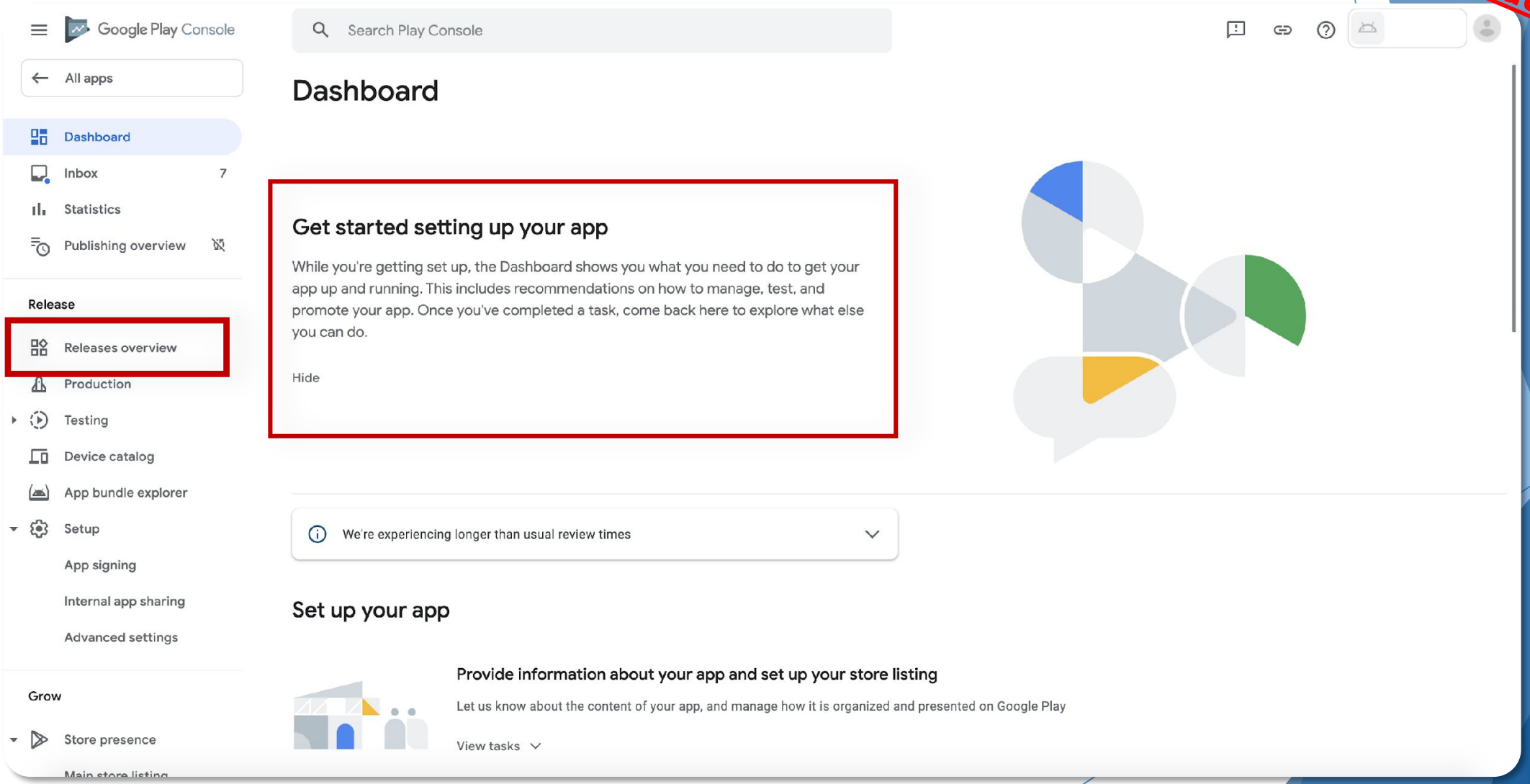
Uploading the app

- We are finally ready to **upload our app to Google Play!**
- We will supply a prepaid Google Play developer account for you, so you should not pay or sign up for an account yourself.



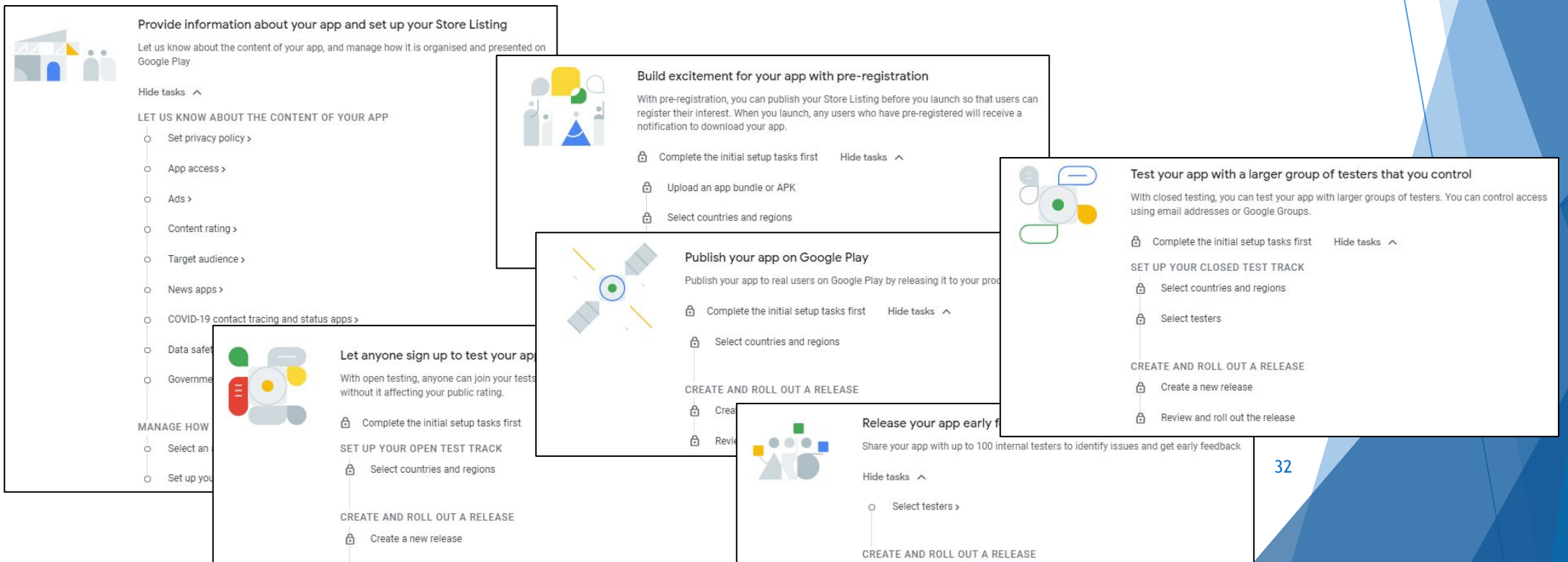
Uploading the app (cont.)

Sprint 1 - Internal Testing
Sprint 2 - Production



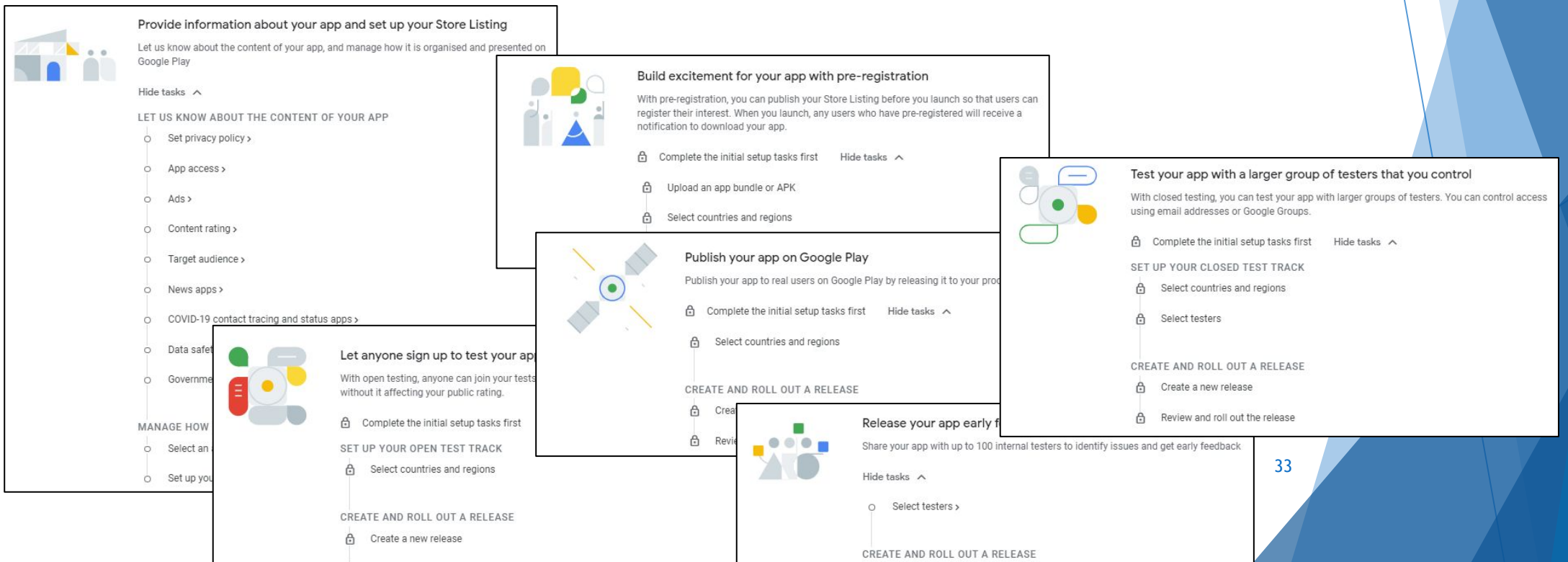
Uploading the app (cont.)

- In the <u>first upload</u> of your app, you'll have to complete many basic forms.
  - Make sure to leave all "Create / Review release" tasks **for the end**!
- Luckily, **you've already prepared almost everything** in advance!



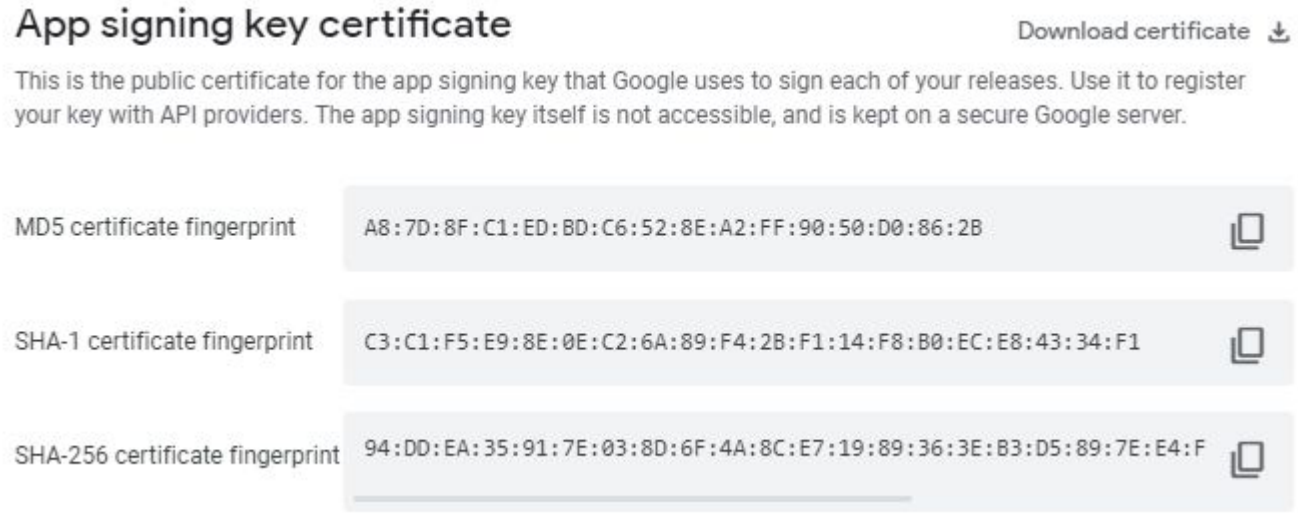
Honorable Mentions

- Data Safety - Explain to Users are you collect and use their data, and how to delete their data from your servers when they Delete Account.
  - You added a Delete Account button, right?
- Content Rating & Target Audience - if your app is targeted to children, the process and review is harder and longer, so it's best to start early.



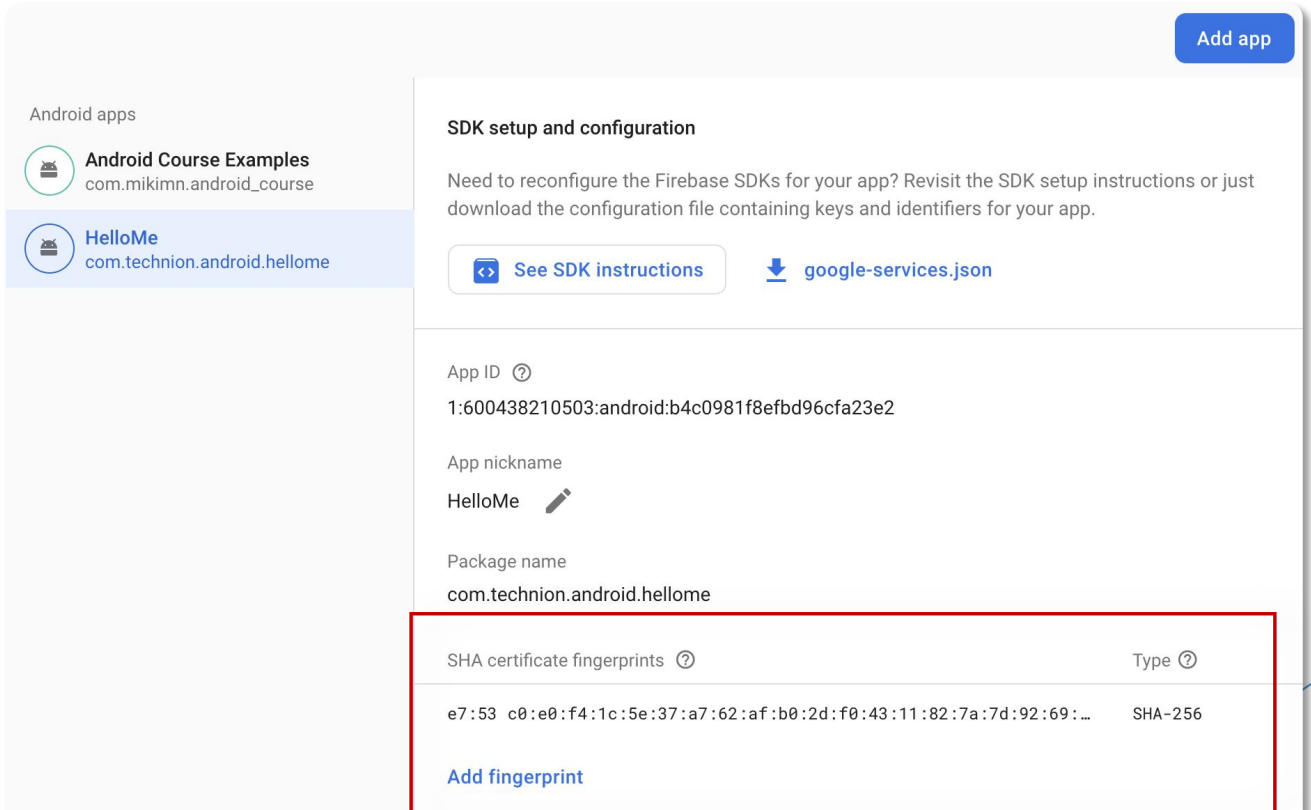
Uploading the app - Fingerprints

- Reminder: Certain APIs use the **app signing certificate fingerprints** to validate your app.
  - Most common example: Firebase!
- You can get the fingerprints from the Google Play Console on the **Release => Setup => App Integrity** page.



Uploading the app – Fingerprints (cont.)

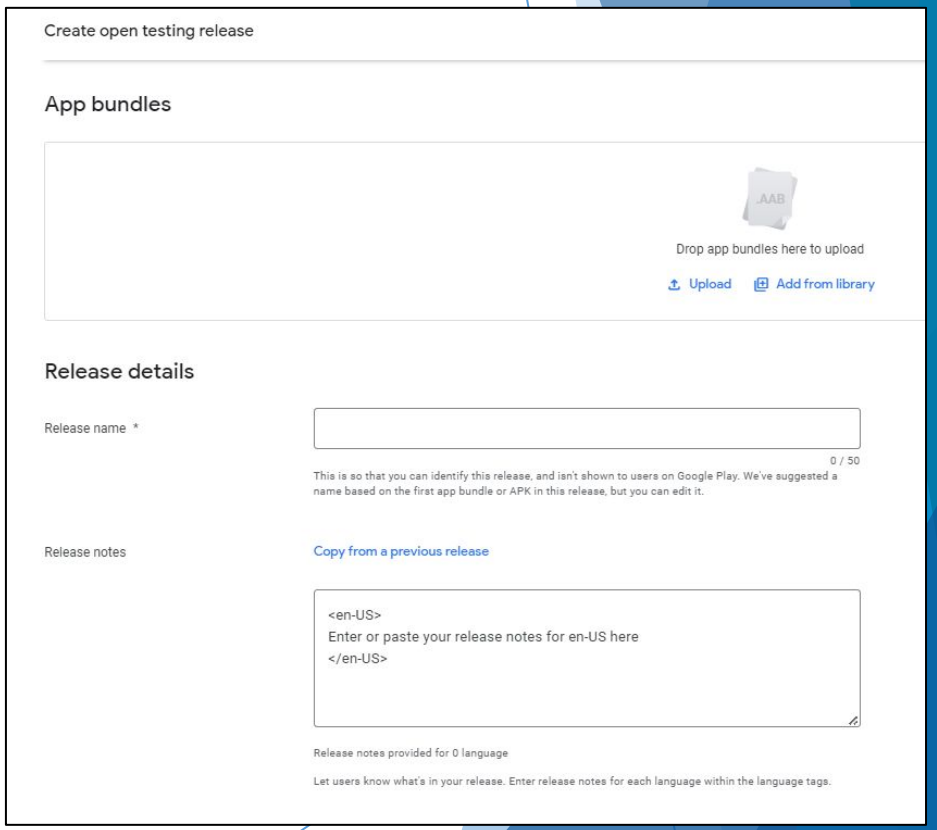
- To add release certificates for **Firebase services**, include them in the fingerprints list.
  - ⚙️ **=> Project Settings => General => Your apps.**



Uploading the app – Create Release

Sprint 1 - Internal Testing
Sprint 2 - Production

- Now that all the forms are complete, it's time to wrap things up and upload our .AAB file.
  - Click on **Release => Internal testing => Create new release**.
  - Upload your .AAB file (might take some time).
  - Make sure to name your release correctly (MYAPP_v1.2.3)
  - Fill up **Release notes** – What's new in this release? (<en-US> for English, <iw-IL> for Hebrew)
  - Click on **Review** and make sure there are no issues, warnings or errors (ignore the translation-related warnings).
  - Click on **Confirm rollout**.
- Invite us as Testers!



Now what?

Hopefully your release will look like this:

Latest releases

Release	Latest version	Track	Release status	Last updated	Countries / regions	Install base	
[redacted] v1.0	1	Internal testing	Available to internal testers Full rollout	Mar 9, 2024 6:17 PM	-	0.00%	→
1 (1.0.0)	2	Production	Available on Google Play Full rollout	Mar 18, 2024 10:41 AM	177 of 177	0.00%	→
1 (1.0.0)	1	Open testing	Draft	-	177 Synced with production	-	→

And not like this:

[redacted]	2	Removed by Google	Update rejected	📌	→

Checklist

- Make sure that **package name** and **Manifest** are correct and updated.
- Set a **new version number and code** in your `pubspec.yaml` file.
- Add an **AboutDialog** to handle copyrights.
- Set a **launcher icon** for your app.
- Generate an **upload key**, reference it from your code and update the **gradle** configurations accordingly.
- **Build your .AAB, .APK files!**
  - At this point, it might be a good idea to install the release APK on your device and check that it works!
    - How would you know that this is the release version and not the debug one?

Checklist (cont.)

- Prepare **graphics** (icon, preview graphic, screenshots…).
- Generate **Privacy Policy & TOS** files and upload them into Gists.
- Enter **Google Play Console** via the invitation sent to you.
- Fill all needed **forms**. For later releases, you might need to refill some of them with updated details (new screenshots, updated TOS…).
- Make sure 3rd party APIs work, by adding **fingerprints** to them.
- Create a **new open testing release**, upload you .AAB and fill details about your new version.
- Make sure that there are no errors, warnings…
- **Send for review and wait a few hours / days!**

So… what did we learn today?

- What are the basic components in an app release.
- How to keep our app installment secure, legal and informative.
- How to combine all of this into a quick-ish uploading process.